«Код Будущего — программирование на Python для специалистов по информационной безопасности»

Модуль 2. Файлы, данные, простая обработка и визуализация.

ТЕМА Работа с данными в формате CSV: изучение отчёта сканера уязвимостей

Практическое занятие

Цель:

Анализировать данные из файлов..

Задачи:

Разберемся: что такое csv-файлы

Научимся считывать из них информацию и записывать данные в файл.

Методические указания и инструкции по выполнению задания

1. **Вспомните и повторите теорию, необходимую для выполнения практического задания**

CSV (Comma-Separated Values) — текстовый формат для хранения табличных данных.

Столбцы разделяются запятыми, точками с запятой или табуляцией.

Первая строка — заголовки.

Применение в кибербезопасности:

Отчёты сканеров уязвимостей (Nessus, OpenVAS).

Экспорт данных из SIEM-систем.

Чтение и запись CSV в Python:

Модуль csv:

csv.reader — построчное чтение.

csv.DictReader — доступ к данным по названиям столбцов.

csv.writer / DictWriter — запись данных.

Параметры: delimiter, quotechar, encoding.

Пример: Анализ отчёта сканера уязвимостей:

Фильтрация строк по уровню риска (Critical, High).

Экспорт результатов в новый CSV-файл.

Проблемы:

Кавычки и переносы строк в ячейках.

Разные кодировки (UTF-8, Windows-1251).

2. **Разберитесь в том, как выполняется задание**

Пример программы для чтения CSV-файла и вывода его содержимого.

Задание 1

Подсчет статистики: найти, сколько уязвимостей у каждого IP

Описание для тренажера:

Напишите программу в файле, которая подсчитывает количество уязвимостей для каждого IP-адреса по данным из файла vulns.log.

Требования к программе:

1. Программа не должна запрашивать ввод у пользователя.
   Она должна сразу начать обработку файла vulns.log, который находится в той же директории.

2. Программа должна открыть файл vulns.log в кодировке utf-8 и построчно прочитать его содержимое.

3. В каждой строке файла первым элементом является IP-адрес.
   Строка может содержать дополнительные данные, но они игнорируются.
   IP-адрес — это первое слово в строке.

4. Нужно подсчитать, сколько раз каждый IP встречается в файле.
   Подсчёт выводится в формате:
   IP: количество
   Выводить каждую строку через print.

5. Пустые строки или строки без данных должны пропускаться.

6. Если файл vulns.log отсутствует:
   • вывести сообщение: Файл vulns.log не найден
   • программа не должна падать с исключением
   • stderr должен быть пустым

7. Все сообщения должны выводиться только через print. stderr пустой.

8. Порядок вывода IP-адресов может быть любым.

9. Название файла программы строго index.py.
   Имя файла, из которого читаются данные, строго vulns.log.

Пример корректного решения:

```
def count_vulns(log_file="vulns.log"):
    counts = {}

    try:
        with open(log_file, "r", encoding="utf-8") as f:
            for line in f:
                parts = line.strip().split()
                if not parts:
                    continue
                ip = parts[0]
                counts[ip] = counts.get(ip, 0) + 1

        for ip, total in counts.items():
            print(f"{ip}: {total}")

    except FileNotFoundError:
        print(f"Файл {log_file} не найден")

if __name__ == "__main__":
    count_vulns()
```

Тренажер: 20301-Counting-statistics

Задание 2

Вывести топ-3 IP с наибольшим числом уязвимостей.

Описание для тренажера:

Напишите программу, которая выводит топ-3 IP-адреса с наибольшим количеством уязвимостей на основе файла vulns.log.

Требования:

1. Программа не должна ничего запрашивать у пользователя.
   Она должна сразу начать обработку файла vulns.log, который расположен в той же директории.

2. Программа должна открыть файл vulns.log в кодировке utf-8 и построчно прочитать его содержимое.

3. В каждой строке файла первым элементом является IP-адрес.
   Это первое слово в строке. Остальная часть строки не используется.

4. Нужно подсчитать, сколько раз каждый IP встречается в файле, и вывести три IP-адреса, которые встречаются чаще всего.

5. Вывод должен содержать ровно три строки — топ-3 IP по частоте.
   Формат строки:
   IP: количество

6. Порядок вывода строго соответствует убыванию количества уязвимостей — первый (самый частый), затем второй, затем третий.
   При равенстве количества порядок определяется внутренней сортировкой Counter.most_common(), то есть тест ожидает именно такой порядок.

7. Пустые строки должны игнорироваться.

8. Если файл vulns.log отсутствует:
   — вывести сообщение: Файл vulns.log не найден

— программа не должна падать
— stderr должен быть пустым

9. Программа должна печатать результат только через print. stderr всегда пустой.

10. Название файла программы строго index.py. Имя обрабатываемого файла строго vulns.log.

Пример корректного решения:

```
from collections import Counter

def top3_ips(log_file="vulns.log"):
    try:
        ips = []
        with open(log_file, "r", encoding="utf-8") as f:
            for line in f:
                parts = line.strip().split()
                if parts:
                    ips.append(parts[0])

        counter = Counter(ips)
        for ip, count in counter.most_common(3):
            print(f"{ip}: {count}")

    except FileNotFoundError:
        print(f"Файл {log_file} не найден")

if __name__ == "__main__":
    top3_ips()
```

Тренажер: 20302-withdraw-ip

Задание 3

Фильтрация и экспорт

Сохранить в новый файл уязвимости с уровнем **Critical**.

Описание для тренажера:

Есть файл **security.log**. В нём записаны разные уязвимости, и у каждой есть уровень опасности: Low, Medium, High или Critical.

Задача — **создать новый файл с названием *critical.log*** и **перенести в него только те строки из security.log, где встречается слово *Critical*.**

Что нужно сделать:

1. Открыть файл **security.log** (он уже существует — его создаёт система).

2. Прочитать его построчно.

3. Каждый раз, когда в строке есть слово **Critical**, записывать эту строку в новый файл **critical.log**.

4. Когда всё будет готово, вывести на экран сообщение:
   Готово

5. Если файла **security.log** почему-то нет, нужно вывести сообщение:
   Файл security.log не найден

Пример корректного решения:

```
def extract_critical(src="security.log",
dst="critical.log"):
    try:
```

```
        with open(src, "r", encoding="utf-8") as f,
open(dst, "w", encoding="utf-8") as out:
            for line in f:
                if "Critical" in line:
                    out.write(line)
        print("Готово")
    except FileNotFoundError:
        print(f"Файл {src} не найден")



if __name__ == "__main__":
    extract_critical()
```

Тренажер: 20303-filtering-and-export

Задание 4

Прочитать логи и определить, какие уязвимости (по CVE) встречаются чаще всего

Описание для тренажера:

Есть файл **security.log**. В нём записаны разные строки, и в них могут встречаться коды уязвимостей, которые начинаются так:

`CVE-год-номер`

`Например: CVE-2021-1111`

Задача — **прочитать этот файл и посчитать, какие CVE встречаются чаще всего.**

Что нужно сделать:

1. Открыть файл **security.log** (он уже будет создан системой).

2. Найти в каждой строке все упоминания CVE по такому виду:
   CVE-XXXX-YYYY, где
   – XXXX — это 4 цифры года,
   – YYYY — это номер от 4 до 7 цифр.

3. Собрать все найденные CVE.

4. Если не найдено ни одного — вывести:
   CVE не найдены

Если они есть — нужно вывести **ТОП-5 самых частых**, по одной строке:

`CVE:количество`

5. Например:
   `CVE-2021-1111:3`

6. Если файл **security.log** отсутствует, вывести:
   Файл security.log не найден

<u>Пример корректного решения:</u>

```
from collections import Counter
import re
def analyze_cve(log_file="security.log"):
    Анализирует логи и находит наиболее частые CVE
    cve_pattern = re.compile(r'CVE-\d{4}-\d{4,7}')
    cve_list = []

    try:
        with open(log_file, 'r') as f:
```

```
        for line in f:
            cve_list.extend(cve_pattern.findall(line))

    if not cve_list:
        print("CVE не найдены")
        return

    cve_counts = Counter(cve_list)
    print("Наиболее частые уязвимости (CVE):")
    for cve, count in cve_counts.most_common(5):
        print(f"{cve}: {count} раз")

  except FileNotFoundError:
    print(f"Файл {log_file} не найден")
# Пример использования
analyze_cve()
```

Пример корректного решения:

```
from collections import Counter
import re


def analyze_cve(log_file="security.log"):
    cve_pattern = re.compile(r"CVE-\d{4}-\d{4,7}")
    cve_list = []

    try:
        with open(log_file, "r", encoding="utf-8") as f:
            for line in f:

cve_list.extend(cve_pattern.findall(line))
```

```
        if not cve_list:
            print("CVE не найдены")
            return

        counts = Counter(cve_list).most_common(5)

        for cve, count in counts:
            print(f"{cve}:{count}")
    except FileNotFoundError:
        print(f"Файл {log_file} не найден")



if __name__ == "__main__":
    analyze_cve()
```

Тренажер: 20304-read-logs

Задание 5

Часто повторяющиеся уязвимости

- **Цель:** отсортировать уязвимости по убыванию CVSS

Описание задание для тренажера:

1. Прочитать файл security.log.
2. В каждой строке найти уязвимость формата CVE-XXXX-YYYY и её оценку CVSS в виде числа с точкой.
3. Собрать все найденные пары CVE и CVSS.
4. Если ни одной такой записи не найдено, вывести сообщение: Нет уязвимостей.

5. Если записи есть, отсортировать их по убыванию значения CVSS (от большего к меньшему).

6. Записать отсортированные данные в файл sorted.log, по одной строке в формате: CVE CVSS.

7. После успешной записи вывести сообщение: Готово.

8. Если файл security.log отсутствует, вывести сообщение: Файл security.log не найден.

Пример корректного решения:

```
import re

def sort_by_cvss(src="security.log", dst="sorted.log"):
    pattern =
re.compile(r"(CVE-\d{4}-\d{4,7}).*?CVSS:(\d+\.\d+)")

    vulns = []

    try:
        with open(src, "r", encoding="utf-8") as f:
            for line in f:
                m = pattern.search(line)
                if m:
                    cve = m.group(1)
                    cvss = float(m.group(2))
                    vulns.append((cve, cvss))

        if not vulns:
            print("Нет уязвимостей")
```

```
            return

        vulns.sort(key=lambda x: x[1], reverse=True)

        with open(dst, "w", encoding="utf-8") as out:
            for cve, cvss in vulns:
                out.write(f"{cve} {cvss}\n")

        print("Готово")
    except FileNotFoundError:
        print(f"Файл {src} не найден")


if __name__ == "__main__":
    sort_by_cvss()
```

Тренажер: 20305-recurring-vulnerabilities

Критерии оценивания

Критерий	Баллы	Требования
1. Корректность работы программы	0-1	– 1 балл: программа работает без ошибок, полностью соответствует условиям задачи, и выводит правильный ответ. – 0 баллов: программа не работает или выдает критичные ошибки, выводит неправильный ответ.
2. Использование необходимых функций	0-2	– 2 балла: при решении использованы все требуемые необходимые языковые средства – 1 балл: при решении использованы не все необходимые языковые средства

		– 0 баллов: требуемые языковые средства не использованы

Шкала оценивания:

Зачтено — 3 балла

Не зачтено — 0 – 2 балла

Требования к оформлению результата выполнения задания: решение задачи предусматривает ввод программного кода в специальное поле тренажера.